

KUBERNETES - TASK -03

TASK 1 — Create a ClusterIP Service for Apache?

```
ubuntu@k8s-master:~$ vi apache-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f apache-pod.yaml
pod/apache created
ubuntu@k8s-master:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
apache    1/1     Running   0          14s
ubuntu@k8s-master:~$ kubectl get pods --show-labels
NAME      READY   STATUS    RESTARTS   AGE   LABELS
apache    1/1     Running   0          66s   app=apache
```

```
apiVersion: v1
kind: Pod
metadata:
  name: apache
  labels:
    app: apache
spec:
  containers:
  - name: apache
    image: httpd:latest
    ports:
    - containerPort: 80
```

This is very important.

We are attaching a label:

app = apache

Later the **Service** will use this label to **find the Pod**.

```
apiVersion: v1
kind: Service
metadata:
  name: apache-service
spec:
  type: ClusterIP
  selector:
    app: apache
  ports:
  - port: 80
    targetPort: 80
```

Very important: selector

Look carefully:

Pod:

labels:

app: apache

Service:

selector:

app: apache

They match.

Service selector

|

| app=apache

↓

Pod label

|

| app=apache

↓

Apache Pod

This is how the Service knows:

"Send traffic to this Pod."

port vs targetPort

This is very important for interviews.

ports:

- port: 80

targetPort: 80

port

The port exposed by the Service.

targetPort

The port on the Pod/container.

```
ubuntu@k8s-master:~$ vi apache-service.yaml
ubuntu@k8s-master:~$ kubectl apply -f apache-service.yaml
service/apache-service created
```

```
apiVersion: v1
kind: Service

metadata:
  name: apache-service

spec:
  type: ClusterIP

  selector:
    app: apache

  ports:
    - port: 80
      targetPort: 80
```

```
ubuntu@k8s-master:~$ kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
apache-service      ClusterIP   10.96.68.135  <none>       80/TCP     4m7s
kubernetes          ClusterIP   10.96.0.1     <none>       443/TCP    2d3h

ubuntu@k8s-master:~$ kubectl get endpoints apache-service
NAME                ENDPOINTS          AGE
apache-service      10.244.2.11:80     6m2s
```

This proves that the Service found the Pod.

Interview answer

ClusterIP is the default Kubernetes Service type. It provides an internal stable IP and DNS name for accessing Pods within the cluster. It is not directly accessible from outside the cluster.

Task 2: Expose an Nginx pod externally using a NodePort service.

Step 1: Create Nginx Pod

```
ubuntu@k8s-master:~$ vi nginx-nodeport.yaml
ubuntu@k8s-master:~$ kubectl apply -f nginx-nodeport.yaml
pod/nginx created
ubuntu@k8s-master:~$ kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
apache  1/1     Running   0           31m
nginx   1/1     Running   0           11s
```

```
apiVersion: v1
kind: Pod

metadata:
  name: nginx
  labels:
    app: nginx

spec:
  containers:
    - name: nginx
      image: nginx:latest
      ports:
        - containerPort: 80
```

Step 2: Create NodePort Service

```
nginx 1/1 Running 0 11s
ubuntu@k8s-master:~$ vi nginx-service.yaml
```

```

apiVersion: v1
kind: Service

metadata:
  name: nginx-nodeport

spec:
  type: NodePort

  selector:
    app: nginx

  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080

```

Notice:

type: NodePort

This tells Kubernetes:

Expose this Service through a port on every Kubernetes node.

nodePort

External access:

<Node-IP>:30080

port

Service port:

80

targetPort

Application/Pod port:

80

```

ubuntu@k8s-master:~$ kubectl replace force -f nginx-service.yaml
service/nginx-nodeport replaced
ubuntu@k8s-master:~$ kubectl get svc nginx-nodeport
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
nginx-nodeport      NodePort    10.101.122.140 <none>         80:30080/TCP     26s
ubuntu@k8s-master:~$ kubectl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
k8s-master          Ready     control-plane  2d3h   v1.29.15   172.31.4.10   <none>        Ubuntu 26.04 LTS     7.0.0-1012-aws   containerd://2.3.5
worker01             Ready     <none>      2d3h   v1.29.15   172.31.4.11   <none>        Ubuntu 26.04 LTS     7.0.0-1006-aws   containerd://2.3.5
worker02             Ready     <none>      2d3h   v1.29.15   172.31.4.12   <none>        Ubuntu 26.04 LTS     7.0.0-1006-aws   containerd://2.3.5
ubuntu@k8s-master:~$ kubectl get endpoints nginx-nodeport
NAME                ENDPOINTS          AGE
nginx-nodeport      10.244.1.14:80     4m51s
ubuntu@k8s-master:~$ curl http://172.31.4.10:30080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>

```

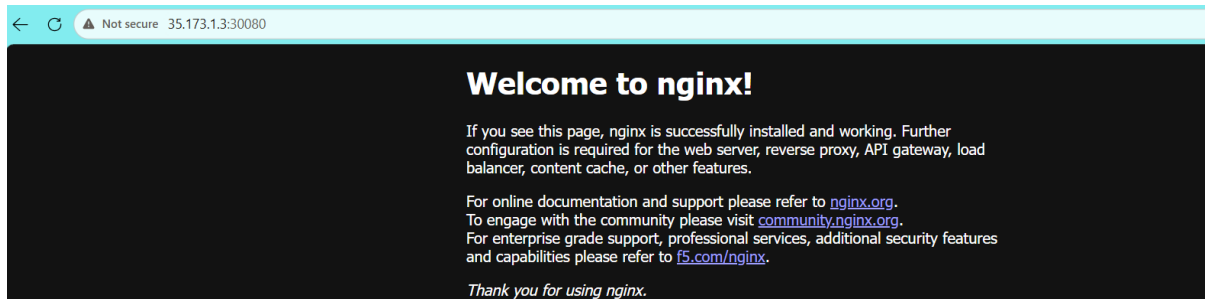
i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 35.173.1.3 PrivateIPs: 172.31.4.85

```
ubuntu@k8s-master:~$ curl ifconfig.me
35.173.1.3ubuntu@k8s-vi nginx-service.yaml
```

Then:

<http://<NODE-IP>:30080>



Task-3 Deploy a ReplicationController to maintain 3 replicas of an Nginx pod.

```
ubuntu@k8s-master:~$ vi nginx-rc.yaml
```

```
apiVersion: v1
kind: ReplicationController

metadata:
  name: nginx-rc

spec:
  replicas: 3

  selector:
    app: nginx-rc

  template:
    metadata:
      labels:
        app: nginx-rc

    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

```
ubuntu@k8s-master:~$ kubectl apply -f nginx-rc.yaml
replicationcontroller/nginx-rc created
```

You should get 3 Pods.

```
ubuntu@k8s-master:~$ kubectl get rc
NAME          DESIRED   CURRENT   READY   AGE
nginx-rc      3         3         3       16s
ubuntu@k8s-master:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
apache                               1/1     Running   0           78m
nginx                                1/1     Running   0           47m
nginx-rc-l6w7w                       1/1     Running   0           38s
nginx-rc-s9j6k                       1/1     Running   0           38s
nginx-rc-v6mr6                       1/1     Running   0           38s
```

What happens if one Pod dies?

```

ubuntu@k8s-master:~$ kubectl delete pod nginx-rc-l6w7w
pod "nginx-rc-l6w7w" deleted
ubuntu@k8s-master:~$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
apache              1/1     Running   0           80m
nginx               1/1     Running   0           48m
nginx-rc-s8g5t      1/1     Running   0           11s
nginx-rc-s9j6k      1/1     Running   0           2m16s
nginx-rc-v6mr6      1/1     Running   0           2m16s
ubuntu@k8s-master:~$

```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 35.173.1.3 PrivateIPs: 172.31.4.85

You'll notice Kubernetes creates a replacement.

Pod 1

Pod 3

Pod 4 ← new

Still:

3 Pods

That's the purpose of a ReplicationController.

Task -4 Scale the ReplicationController from 3 replicas to 5 replicas.

Current:

3 Pods

We want:

5 Pods

Run:

kubectl scale rc nginx-rc --replicas=5

```

ubuntu@k8s-master:~$ kubectl get rc
NAME      DESIRED   CURRENT   READY   AGE
nginx-rc  3         3         3       40m
ubuntu@k8s-master:~$ kubectl scale rc nginx-rc --replicas=5
replicationcontroller/nginx-rc scaled
ubuntu@k8s-master:~$ kubectl get rc
NAME      DESIRED   CURRENT   READY   AGE
nginx-rc  5         5         5       42m
ubuntu@k8s-master:~$

```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 35.173.1.3 PrivateIPs: 172.31.4.85

Task -5: Create a ReplicaSet to manage pods based on multiple labels (prod and test)

1. Create the ReplicaSet

```
kubectl apply -f rs-prod-test.yaml
```

2. Check ReplicaSet

```
kubectl get rs
```

3. Check Pods and labels

```
kubectl get pods --show-labels
```

```
apiVersion: apps/v1
kind: ReplicaSet

metadata:
  name: nginx-prod-test

spec:
  replicas: 3

  selector:
    matchExpressions:
      - key: env
        operator: In
        values:
          - prod
          - test

  template:
    metadata:
      labels:
        app: nginx
        env: prod

    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

```
ubuntu@k8s-master:~$ vi rs-prod-test.yaml
ubuntu@k8s-master:~$ kubectl apply -f rs-prod-test.yaml
replicaset.apps/nginx-prod-test created
ubuntu@k8s-master:~$ kubectl get rs
NAME                DESIRED   CURRENT   READY   AGE
nginx-prod-test      3          3         3       10s
ubuntu@k8s-master:~$ kubectl get pods
NAME                READY     STATUS    RESTARTS   AGE
apache              1/1      Running   0           162m
nginx               1/1      Running   0           130m
nginx-prod-test-lxtb9 1/1      Running   0           19s
nginx-prod-test-m9s66 1/1      Running   0           19s
nginx-prod-test-w8mgw 1/1      Running   0           19s
ubuntu@k8s-master:~$ kubectl get pods --show-labels
NAME                READY     STATUS    RESTARTS   AGE   LABELS
apache              1/1      Running   0           162m   app=apache
nginx               1/1      Running   0           131m   app=nginx
nginx-prod-test-lxtb9 1/1      Running   0           57s   app=nginx, env=prod
nginx-prod-test-m9s66 1/1      Running   0           57s   app=nginx, env=prod
nginx-prod-test-w8mgw 1/1      Running   0           57s   app=nginx, env=prod
```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 35.173.1.3 PrivateIPs: 172.31.4.85

```
ubuntu@k8s-master:~$ kubectl get pods -l env=prod
NAME                READY   STATUS    RESTARTS   AGE
nginx-prod-test-lxtb9 1/1     Running   0           2m15s
nginx-prod-test-m9s66 1/1     Running   0           2m15s
nginx-prod-test-w8mgw 1/1     Running   0           2m15s
ubuntu@k8s-master:~$ kubectl get pods -l env=test
No resources found in default namespace.
ubuntu@k8s-master:~$ kubectl get pods -L env
NAME                READY   STATUS    RESTARTS   AGE   ENV
apache              1/1     Running   0           164m
nginx               1/1     Running   0           132m
nginx-prod-test-lxtb9 1/1     Running   0           2m36s   prod
nginx-prod-test-m9s66 1/1     Running   0           2m36s   prod
nginx-prod-test-w8mgw 1/1     Running   0           2m36s   prod
ubuntu@k8s-master:~$
```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 35.173.1.3 PrivateIPs: 172.31.4.85

matchLabels:

Think:

"Exactly this label."

app: nginx

env: prod

➔ app=nginx AND env=prod

matchExpressions:

Think:

"Give me more control."

key: env

operator: In

values:

- prod

- test

➔ env=prod OR env=test

Interview answer 🎯

matchLabels is used for exact key-value label matching, whereas matchExpressions provides more flexible selection using operators such as In, NotIn, Exists, and DoesNotExist.

One more important point: multiple conditions inside either selector are combined as AND, while values inside In are alternatives (OR).

Task -06: Deploy a ReplicaSet that excludes pods with the label backend.

Here the requirement means:

Create a ReplicaSet that manages Pods which do NOT have the label backend.

Step 1: Create ReplicaSet YAML

vi exclude-backend-rs.yaml

Step 2: Apply YAML

kubectl apply -f exclude-backend-rs.yaml

Step 3: Verify ReplicaSet

kubectl get rs

Step 4: Verify Pods with Labels

kubectl get pods --show-labels

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: exclude-backend-rs
spec:
  replicas: 3
  selector:
    matchExpressions:
      - key: tier
        operator: NotIn
        values:
          - backend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
        - name: nginx
          image: nginx

ubuntu@k8s-master:~$ vi exclude-backend-rs.yaml
ubuntu@k8s-master:~$ kubectl apply -f exclude-backend-rs.yaml
replicaset.apps/exclude-backend-rs created
ubuntu@k8s-master:~$ kubectl get rs
NAME                DESIRED   CURRENT   READY   AGE
exclude-backend-rs  3         3         3       11s
ubuntu@k8s-master:~$ kubectl get pods --show-labels
NAME                                READY   STATUS    RESTARTS   AGE   LABELS
exclude-backend-rs-f7h97            1/1     Running   0           3m48s   tier=frontend
exclude-backend-rs-gtwrt            1/1     Running   0           3m48s   tier=frontend
exclude-backend-rs-p9zk8            1/1     Running   0           3m48s   tier=frontend
ubuntu@k8s-master:~$
```

i-004f01f684ce82ec3 (k8s-master)
PublicIPs: 35.173.1.3 PrivateIPs: 172.31.4.85

Task-07: Test load balancing across multiple pods using a NodePort service.

```
ubuntu@k8s-master:~$ vi nginx-rs.yaml
ubuntu@k8s-master:~$ kubectl apply -f nginx-rs.yaml
replicaset.apps/nginx-rs created
ubuntu@k8s-master:~$ kubectl get pods -l app=nginx --show-labels
NAME                                READY   STATUS    RESTARTS   AGE   LABELS
nginx-rs-885c4                      1/1     Running   0           5s    app=nginx,environment=prod
nginx-rs-cn8nc                      1/1     Running   0           5s    app=nginx,environment=prod
nginx-rs-gv2ln                      1/1     Running   0           5s    app=nginx,environment=prod
ubuntu@k8s-master:~$ vi nginx-nodeport-lb.yaml
ubuntu@k8s-master:~$ kubectl apply -f nginx-nodeport-lb.yaml
service/nginx-lb created
ubuntu@k8s-master:~$ kubectl get svc nginx-lb
NAME    TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
nginx-lb NodePort     10.102.2.123   <none>         80:30090/TCP 6s
ubuntu@k8s-master:~$ kubectl get endpoints nginx-lb
NAME    ENDPOINTS                                         AGE
nginx-lb 10.244.1.23:80,10.244.1.24:80,10.244.2.20:80 26s
ubuntu@k8s-master:~$ curl http://172.31.4.10:30090
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
```

i-004f01f684ce82ec3 (k8s-master)
PublicIPs: 35.173.1.3 PrivateIPs: 172.31.4.85

Steps

1. Created a ReplicaSet with **3 Nginx Pods** using the label app=nginx.

```

apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
      environment: prod
  template:
    metadata:
      labels:
        app: nginx
        environment: prod
    spec:
      containers:
        - name: nginx
          image: nginx:latest

```

2. Verified that all 3 Pods were running:

kubectl get pods -l app=nginx

3. Created a NodePort Service named nginx-lb with:

- Service Port: 80
- NodePort: 30090
- Selector: app=nginx

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-lb
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30090

```

4. Verified the Service:

kubectl get svc nginx-lb

5. Verified that the Service was connected to all 3 Pods:

kubectl get endpoints nginx-lb

The Service showed three Pod IP addresses, confirming that all three Pods were backend endpoints.

6. Accessed the application through the NodePort:

curl <http://172.31.4.10:30090>

```

ubuntu@k8s-master:~$ kubectl get pods -l app=nginx
NAME                READY   STATUS    RESTARTS   AGE
nginx-rs-885c4       1/1     Running   0           22m
nginx-rs-cn8nc       1/1     Running   0           22m
nginx-rs-gv2ln       1/1     Running   0           22m
ubuntu@k8s-master:~$ kubectl exec nginx-exclude-backend-fhk8n -- hostname
Error from server (NotFound): pods "nginx-exclude-backend-fhk8n" not found
ubuntu@k8s-master:~$ kubectl exec nginx-rs-885c4 -- hostname
nginx-rs-885c4
ubuntu@k8s-master:~$ curl http://172.31.4.10:30090
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }

```

To identify which Pod handled the request, different responses were configured in each Nginx Pod. Multiple requests were then sent through the NodePort.

```

ubuntu@k8s-master:~$ kubectl exec nginx-rs-885c4 -- curl -s localhost
Pod-1
ubuntu@k8s-master:~$ kubectl exec nginx-rs-cn8nc -- curl -s localhost
Pod-2
ubuntu@k8s-master:~$ kubectl exec nginx-rs-gv2ln -- curl -s localhost
Pod-3
ubuntu@k8s-master:~$ curl -s http://172.31.4.10:30090
Pod-2
ubuntu@k8s-master:~$ for i in {1..15}; do curl -s http://172.31.4.10:30090; echo; done
Pod-3
Pod-3
Pod-1
Pod-3
Pod-3
Pod-3
Pod-1
Pod-1
Pod-3
Pod-3
Pod-3
Pod-1
Pod-1

```

Result

The NodePort Service successfully connected to all three Nginx Pods and distributed incoming requests among the available Pods.

Conclusion

A Kubernetes Service uses **labels and selectors** to identify backend Pods. The NodePort exposes the Service outside the cluster through port 30090, while Kubernetes distributes requests to the matching Pods.

Task -08: Delete a ReplicationController without affecting the running pods.

Step 1: create rc

Step 2: check the pods created or not

Step 3: delete the rc without affecting pods

kubectl delete rc --cascade=false nginx-rc

Step 4: again check the pods

So still pods is there...task successfully done

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx-rc
spec:
  replicas: 3
  selector:
    app: nginx-rc
  template:
    metadata:
      labels:
        app: nginx-rc
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80

ubuntu@k8s-master:~$ kubectl apply -f nginx-rc.yaml
replicationcontroller/nginx-rc created
ubuntu@k8s-master:~$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
exclude-backend-rs-f7h97            1/1      Running   0           65m
exclude-backend-rs-gtwrt            1/1      Running   0           65m
exclude-backend-rs-p9zk8            1/1      Running   0           65m
nginx-rc-5xvzh                      1/1      Running   0           10s
nginx-rc-jgvkd                      1/1      Running   0           10s
nginx-rc-thxgb                      1/1      Running   0           10s
nginx-rs-885c4                      1/1      Running   0           48m
nginx-rs-cn8nc                      1/1      Running   0           48m
nginx-rs-gv2ln                      1/1      Running   0           48m
ubuntu@k8s-master:~$ kubectl delete rs --all
replicaset.apps "exclude-backend-rs" deleted
replicaset.apps "nginx-rs" deleted
ubuntu@k8s-master:~$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
nginx-rc-5xvzh                      1/1      Running   0           44s
nginx-rc-jgvkd                      1/1      Running   0           44s
nginx-rc-thxgb                      1/1      Running   0           44s
ubuntu@k8s-master:~$ kubectl delete rc --caseade=false nginx-rc
error: unknown flag: --caseade
See 'kubectl delete --help' for usage.
ubuntu@k8s-master:~$ kubectl delete rc --cascade=false nginx-rc
warning: --cascade=false is deprecated (boolean value) and can be replaced with --cascade=orphan.
replicationcontroller "nginx-rc" deleted
ubuntu@k8s-master:~$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
nginx-rc-5xvzh                      1/1      Running   0           118s
nginx-rc-jgvkd                      1/1      Running   0           118s
nginx-rc-thxgb                      1/1      Running   0           118s
ubuntu@k8s-master:~$ kubectl describe nginx-rc
error: the server doesn't have a resource type "nginx-rc"
```

i-004f01f684ce82ec3 (k8s-master)

PublicIPs: 35.173.1.3 PrivateIPs: 172.31.4.85

```
ubuntu@k8s-master:~$ kubectl get rc
No resources found in default namespace.
```